

Packing Squares into Rectangles: Comparison of CP, SAT and ASP solutions

Luca Davì 75214

NOVA School of Science and Technology

1 Problem Description

Rectangle Packing problems involve packing all squares with sizes 1×1 to $n \times n$ into an enclosing rectangle of minimum area [4]. Rectangle packing has applications in a variety of real-world settings, e.g. packing of blocks into a circuit layout and scheduling.

The objective of this report is to illustrate the modeling and implementation of the Rectangle Packing problem under three distinct formal frameworks: as a propositional satisfiability (SAT) instance using PySAT [3], as an Answer Set Programming (ASP) instance relying exclusively on native language constructs, and as an ASP instance augmented with external control via the Clingo Python API[2]. For each formulation, we discuss the choice of constraints, symmetry breaking, and implementation strategies, and evaluate their impact on solver performance.

2 SAT Model

We used the established constraint model for the rectangle packing problem [4]. Each square to be placed is defined by a pair of decision variables X and Y , and two integer constants W and H representing its dimensions. In the case of packing squares, W and H are identical.

The SAT instance is obtained by translating these integer domains into propositional logic via the order encoding, which introduces one Boolean variable per integer value in each domain:

2.1 Variables Definitions and Domains

For each square $i \in \{1, \dots, n\}$ of size $i \times i$, we define two integer decision variables:

- $x_i \in \{0, \dots, \text{maxW} - i\}$ - the x-coordinate of the bottom-left corner,
- $y_i \in \{0, \dots, \text{maxH} - i\}$ - the y-coordinate of the bottom-left corner.

The upper bounds $\text{maxW} - i$ and $\text{maxH} - i$ enforce that each square lies entirely within the rectangle. The widths and heights are fixed parameters $w_i = h_i = s_i = i$.

The encoding chosen for the SAT instance uses the ordering encoding, introducing one Boolean variable per integer value in each domain:

$$\begin{aligned} ox(i, v) &\iff x_i \leq v, \\ oy(i, v) &\iff y_i \leq v. \end{aligned}$$

2.2 Constraints

Ordering variables Consistency Chain The introduction of ordering variables requires imposing the implication chain $x_i \leq v - 1 \implies x_i \leq v$ for every value v in the domain, and analogously for y_i . Helper functions return the domain limits for each square i :

$$\max_x(i) = \max W - i, \quad \max_y(i) = \max H - i.$$

These yield the following clauses:

- $ox(i, \max_x(i))$,
- $oy(i, \max_y(i))$,
- $\neg ox(i, v - 1) \vee ox(i, v) \quad \forall v \in [1, \max_x(i)]$,
- $\neg oy(i, v - 1) \vee oy(i, v) \quad \forall v \in [1, \max_y(i)]$.

Non-Overlapping Constraints Non-overlapping conditions are introduced using indicator variables $dv_{i,j,d}$ with $d \in \{l, r, t, b\}$ indicating that square i is, respectively, to the left of, to the right of, above, or below square j .

For each pair i, j , the following indicator variables are defined:

- $l \iff (x_i + s_i \leq x_j)$ (square i is to the left of square j),
- $r \iff (x_j + s_j \leq x_i)$ (square i is to the right of square j),
- $b \iff (y_i + s_i \leq y_j)$ (square i is below square j),
- $t \iff (y_j + s_j \leq y_i)$ (square i is above square j).

At least one of these four directions must hold for every pair, enforced by the disjunction clause:

$$l \vee r \vee t \vee b.$$

Each indicator variable implies a corresponding inequality, which is in turn encoded via the ordering variables. The resulting implication clauses are:

- $\neg l \vee \neg ox(j, v) \vee ox(i, v - s_i)$ $\forall v \in [0, W - s_j]$,
encoding $l \Rightarrow x_i + s_i \leq x_j$,
- $\neg r \vee \neg ox(i, v) \vee ox(j, v - s_j)$ $\forall v \in [0, W - s_i]$,
encoding $r \Rightarrow x_j + s_j \leq x_i$,
- $\neg b \vee \neg oy(j, v) \vee oy(i, v - s_i)$ $\forall v \in [0, H - s_j]$,
encoding $b \Rightarrow y_i + s_i \leq y_j$,
- $\neg t \vee \neg oy(i, v) \vee oy(j, v - s_j)$ $\forall v \in [0, H - s_i]$,
encoding $t \Rightarrow y_j + s_j \leq y_i$.

The clauses are generated for all v in the stated range; when $v - s_i < 0$ or $v - s_j < 0$ the third literal is omitted, as the implied inequality holds trivially.

2.3 Symmetry Breaking Constraints

A rectangle of dimensions $\text{maxW} \times \text{maxH}$ presents four symmetric solutions: the original placement, a horizontal reflection, a vertical reflection, and their composition. To eliminate redundant solutions, the largest square is restricted to the bottom-left quadrant of the rectangle by imposing

$$ox(n-1, 0) \wedge oy(n-1, 0)$$

which forces $x_n \leq 0$ and $y_n \leq 0$, pinning the largest square to the bottom-left corner and eliminating all four reflective symmetries.

3 ASP Model

Both ASP formulations share the same core encoding for variable domains, non-overlapping constraints, and symmetry breaking. They differ only in how the objective of minimising the enclosing rectangle area is handled: the first delegates that responsibility to an external Python controller, the second expresses it using a weak constraint.

3.1 Variables Definitions and Domains

For each square $i \in \{1, \dots, n\}$, the position of its bottom-left corner is represented by exactly one ground fact from each of the following choice rules:

```
{ pos_X(I, 0..wmax-I) } = 1 :- square(I).
{ pos_Y(I, 0..hmax-I) } = 1 :- square(I).
```

The cardinality constraint $= 1$ enforces that a unique value is chosen for each coordinate, directly encoding the integer domain $x_i \in \{0, \dots, \text{wmax} - i\}$ and $y_i \in \{0, \dots, \text{hmax} - i\}$. The constants `wmax` and `hmax` are passed at grounding time and act as loose upper bounds on the rectangle dimensions.

3.2 Constraints

Non-Overlapping Two squares I and J overlap if and only if they intersect on both the x - and y -axes simultaneously. The constraint is therefore encoded as a single integrity constraint that forbids any model in which all four axis-overlap conditions hold at once:

$$\perp \leftarrow \begin{cases} x_I + I > x_J, \\ x_J + J > x_I, \\ y_I + I > y_J, \\ y_J + J > y_I. \end{cases}$$

Any valid placement must satisfy at least one separation condition along either axis, which corresponds exactly to the four-direction disjunction of the constraint model.

Symmetry Breaking The same symmetry-breaking constraint as in the SAT model is used: the largest square n is forced to the bottom-left corner by the integrity constraints:

$$\perp \leftarrow pos_X(n, X), X > 0, \quad \perp \leftarrow pos_Y(n, Y), Y > 0.$$

ASP with External Control The first formulation does not express the minimisation objective within the logic problem. Instead, the rectangle dimensions are communicated to the grounder via two external atoms definitions [1]:

```
#external ext_w(1..wmax).
#external ext_h(1..hmax).
```

For a candidate rectangle $W \times H$, the controller activates exactly `ext_w(W)` and `ext_h(H)`, and the following integrity constraints enforce that no square violates the active bounds:

$$\begin{aligned} \perp &\leftarrow pos_X(I, X), ext_w(W), X + I > W, \\ \perp &\leftarrow pos_Y(I, Y), ext_h(H), Y + I > H. \end{aligned}$$

The program is grounded once and the solver is called incrementally; between calls the previously active external atoms are retracted and the new ones are asserted, avoiding the overhead of re-grounding.

ASP with Native Minimisation The second formulation replaces the external control loop with a native weak constraint. The bounding box of the packing is computed by deriving the occupied width and height as aggregate expressions:

```
width(W)  :- W = #max { X+I : pos_X(I,X), square(I); 0 }.
height(H) :- H = #max { Y+I : pos_Y(I,Y), square(I); 0 }.
area(A)   :- width(W), height(H), W > 0, H > 0, A = W*H.
#minimize { A@1 : area(A) }.
```

The `#max` aggregate scans all placed squares and returns the rightmost (resp. topmost) occupied coordinate, yielding the tightest enclosing rectangle for any given placement. The `#minimize` statement instructs Clingo’s built-in optimisation to search for the model minimising $A = W \times H$ at priority level 1.

4 Implementation

4.1 SAT Model Implementation

The SAT instance is constructed by the `build_cnf(n,W,H)` function, which takes the number of squares n and the candidate rectangle dimensions $W \times H$ as parameters and returns a CNF object together with the variable pool.

Variable Encoding Boolean variables are managed through PySAT’s `IDPool`, which assigns a unique integer identifier to each named variable on demand. Ordering variables are registered as `ox_i_v` and `oy_i_v`, and direction indicator variables as `dv_i_j_d`, where $d \in \{l, r, b, t\}$.

Clause Generation Clauses are appended incrementally to the CNF object. The consistency chain and non-overlapping constraints are generated as described in Section 2, iterating over all squares for the domain constraints and over all pairs $i < j$ via `itertools.combinations` for the non-overlapping clauses. Symmetry breaking is enforced by appending the two unit clauses $[ox(n-1, 0)]$ and $[oy(n-1, 0)]$ after all other constraints have been added.

Solving and Decoding The constructed CNF is handed to a `Glucose42` solver instance. On satisfiability, the model is retrieved as a set of signed integer literals and the integer coordinates are recovered by exploiting the ordering encoding directly: since $ox(i, v) \Leftrightarrow x_i \leq v$, the value of x_i is the smallest v for which the corresponding literal appears positively in the model, or $\max_x(i)$ if none does. Decoding y_i follows the same principle. The solver is invoked once per candidate rectangle and is embedded in the dichotomic search described in Section 4.4.

4.2 ASP with External Control Implementation

The `Solver` class wraps a single `clingo.Control` instance that is constructed and grounded once at initialisation. Each call to `Solver.solve(W,H)` retracts the previous `ext_w` and `ext_h` atoms via `assign_external(..., False)`, asserts the new ones via `assign_external(..., True)`, and invokes the solver, collecting the first model found before breaking.

The search strategy over candidate dimensions is shared with the SAT implementation and is described in Section 4.4.

4.3 ASP with Native Minimisation Implementation

The controller is minimal: a single `clingo.Control` instance is constructed, the program is loaded and grounded once, and `ctl.solve(yield_=True)` is called. The handler iterates over improving models, printing the area of each intermediate solution, and terminates once Clingo certifies optimality. The final model holds the globally minimal area together with the corresponding `pos_X` and `pos_Y` facts. Unlike the external-control formulation, no search loop is implemented in Python; the entire optimisation is delegated to Clingo’s internal procedure.

4.4 Dichotomic Search Strategy

The SAT model (Section 2) and the ASP model with external control (Section 3.2) act as decision procedures for a fixed candidate rectangle $W \times H$ rather

than expressing an optimisation objective internally. The minimisation of the enclosing area is delegated to an external search loop shared between the two implementations.

Outer loop over widths. Let $A_{\min} = \sum_{i=1}^n i^2$ be the total area occupied by all squares, which provides a lower bound on any feasible rectangle. The width W is iterated from n up to $\sum_{i=1}^n i$, corresponding respectively to the minimum width that can accommodate the largest square and the case in which all squares are placed in a single row. Since the height must satisfy $H \geq n$ to fit the largest square, the area of any rectangle with the current width is bounded below by $W \cdot n$. The outer loop therefore terminates as soon as $W \cdot n \geq A^*$, where A^* is the best area found so far, since no further width can yield a strict improvement.

Inner dichotomic search over heights. For each candidate width W , feasible heights are bounded by:

$$H_{\text{lb}} = \max\left(n, \left\lceil \frac{A_{\min}}{W} \right\rceil\right), \quad H_{\text{ub}} = \min\left(W, \left\lfloor \frac{A^* - 1}{W} \right\rfloor\right).$$

The lower bound combines the minimum height needed for the largest square with the area lower bound. The upper bound caps the height at W to avoid redundant non-square rectangles and enforces a strict improvement over the current best by requiring $W \cdot H < A^*$, i.e. $H \leq \lfloor (A^* - 1)/W \rfloor$. If $H_{\text{lb}} > H_{\text{ub}}$ the current width cannot yield an improvement and is skipped without invoking the solver.

When the bounds are feasible, a binary search is performed over $[H_{\text{lb}}, H_{\text{ub}}]$: the midpoint H_{mid} is submitted to the solver, a SAT result tightens the upper bound to $H_{\text{mid}} - 1$ and records the solution, while an UNSAT result raises the lower bound to $H_{\text{mid}} + 1$. The smallest feasible height for the current width is thus found in $O(\log H)$ solver calls rather than $O(H)$.

5 Experimental Results

All experiments were conducted on an Apple Macbook Air (M4, 2025), running Python 3.14.5, Clingo 5.8.0, PySAT 1.9.

5.1 Solution Quality

All three implementations find the same minimum enclosing area for every instance up to $n = 10$ (Table 1), and the SAT model agrees with the CP-certified optimum across the entire shared range $n = 5$ to $n = 17$, including identical rectangle dimensions in each case. Fill percentages remain consistently high, reflecting the increasingly tight packing that the solver is forced to find as the problem grows.

Table 1: Results for $n = 5$ to 11. ASP-min times split as grounding + solving. ASP-ext and ASP-min were aborted at $n = 11$ (time limits of 13 min and 12 min respectively).

n	$W \times H$	Area	Fill (%)	SAT		ASP-ext		ASP-min	
				t_{sat} (s)	t_{tot} (s)	t (s)	t_{gnd} (s)	t_{slv} (s)	
5	12×5	60	91.7	0.001	0.002	0.01	0.010	0.004	
6	11×9	99	91.9	0.001	0.003	0.02	0.043	0.021	
7	14×11	154	90.9	0.002	0.009	0.08	0.195	0.172	
8	15×14	210	97.1	0.001	0.007	0.32	0.613	0.803	
9	20×15	300	95.0	0.004	0.028	3.43	1.913	22.383	
10	27×15	405	95.1	0.003	0.055	41.20	5.589	264.619	
11	27×19	513	98.6	0.004	0.050	aborted			

Table 2: SAT results for $n = 12$ to 20 against the CP baseline (available up to $n = 17$). All optimal areas agree where both are defined.

n	W	H	Area	SAT		W	H	Area	CP	
				t (s)					t (s)	
12	29	23	667	0.106		29	23	667	0.112	
13	38	22	836	0.189		38	22	836	0.182	
14	45	23	1035	0.288		45	23	1035	1.689	
15	55	23	1265	0.820		55	23	1265	5.359	
16	54	28	1512	1.408		54	28	1512	29.978	
17	46	39	1794	2.635		46	39	1794	47.403	
18	69	31	2139	29.390					—	
19	53	47	2491	104.823					—	
20	85	34	2890	304.431					—	

5.2 Scalability

The three implementations differ sharply in how far they scale.

The SAT model scales to $n = 20$ within five minutes, benefiting from the compact ordering encoding and the efficiency of the **Glucose42** solver.

The ASP model with external control shares the same dichotomic search strategy as the SAT model but is substantially slower at every instance. At $n = 10$ it requires 41 seconds against the SAT model’s 55 milliseconds. At $n = 11$ it finds the optimal solution but cannot prove its optimality within 13 minutes and is aborted.

Unlike the SAT model, which constructs and solves a fresh CNF for each candidate rectangle, the ASP model grounds the program once over the full domain at startup, and the Clingo solver must then re-propagate the entire grounded theory on each incremental call, creating a compounding overhead.

The ASP model with native minimisation exhibits a different bottleneck: grounding is fast relative to solving (5.6 s versus 246.6 s at $n = 10$), but the internal branch-and-bound search over the area objective becomes intractable. At

$n = 11$ the solver likewise finds the optimal solution, but cannot prove its optimality within 13 minutes. The difficulty lies not in finding an optimal placement, but proving that no smaller rectangle exists.

5.3 Comparison with CP

For the instances covered by both SAT and CP (Table 2), the SAT model is consistently faster from $n = 13$ onward. At $n = 15$ the gap is one order of magnitude (0.82s versus 5.36s); at $n = 17$ it reaches roughly $18\times$ (2.64s versus 47.40s). The CP baseline uses the dom/wdeg variable-ordering heuristic, which performs well within each individual **pack** call but does not exploit the dichotomic search structure applied externally to the SAT and ASP-ext models. The SAT model benefits from the combination of an efficient propositional encoding, a state-of-the-art CDCL solver, and the logarithmic reduction in solver calls afforded by the dichotomic search.

For $n \leq 12$, however, the CP model is faster, often by an order of magnitude. At these small sizes the overhead of building and solving multiple CNF instances outweighs the cost of a single CP solve, and the dom/wdeg heuristic is sufficient to prune the search space efficiently.

The solution for $n = 17$ is shown in figure 1.

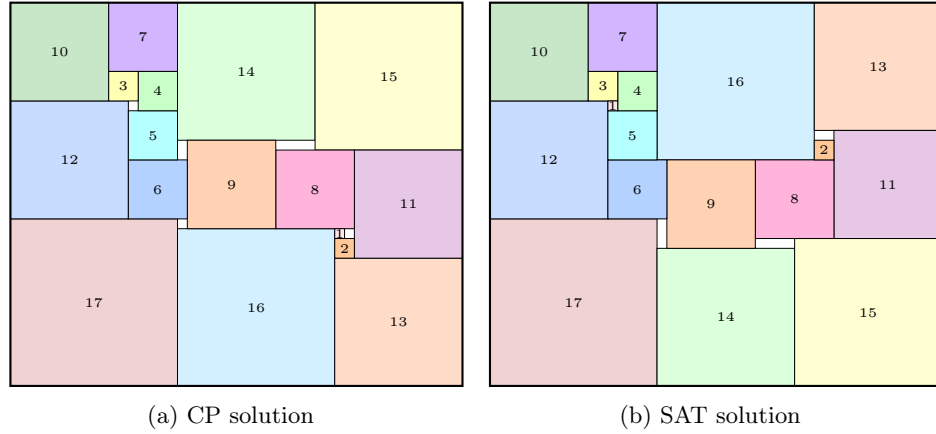


Fig. 1: Two optimal packings for $n = 17$ in a 46×39 rectangle (area = 1794, fill = 99.5%). Both achieve the same minimum area with distinct square arrangements, illustrating the multiplicity of optimal solutions at this instance size.

6 Conclusions

In this report we presented three models for the Rectangle Packing problem: a SAT instance using PySAT with the Glucose42 solver, and two ASP instances

using Clingo, one driven by an external Python controller and one relying on native minimisation via weak constraint.

The SAT model encodes integer coordinates through an order encoding, reducing non-overlap to a set of implication clauses over Boolean ordering variables. Both the SAT model and the ASP model with external control delegate area minimisation to a dichotomic search strategy, which reduced the number of solver calls from $O(H)$ to $O(\log H)$ per candidate width. The ASP model with native minimisation instead delegates the entire optimisation to Clingo’s internal branch-and-bound procedure.

The SAT model scales to $n = 20$ within five minutes and matches the CP-certified optimum on all shared instances, consistently outperforming the CP baseline from $n = 13$ onward. The ASP model with external control finds correct solutions but is substantially slower at every instance, becoming impractical beyond $n = 10$; this is attributable to the overhead of re-propagating a fully grounded theory on each incremental solver call. The ASP model with native minimisation is limited by the scalability of Clingo’s search: it finds optimal solutions up to $n = 10$, but cannot certify optimality at $n = 11$ within the time limit, revealing that the bottleneck lies in proving optimality rather than in finding an optimal solution.

Symmetry breaking, applied uniformly across all three models by fixing the largest square to the bottom-left corner, allowed to keep the search space tractable.

7 Usage of AI Declaration

The author of this report used Claude(Anthropic) for debugging implementation code, generating visualisation and statistics-collection code, and revising selected passages of this report for grammar and flow. All generated or revised content was reviewed and verified by the author.

References

1. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Clingo= asp+ control: Preliminary report. arXiv preprint arXiv:1405.3694 (2014)
2. Gebser, M., Kaminski, R., Kaufmann, B., Schaub, T.: Multi-shot asp solving with clingo (2018), <https://arxiv.org/abs/1705.09811>
3. Ignatiev, A., Morgado, A., Marques-Silva, J.: PySAT: A Python toolkit for prototyping with SAT oracles. In: SAT. pp. 428–437 (2018). https://doi.org/10.1007/978-3-319-94144-8_26, https://doi.org/10.1007/978-3-319-94144-8_26
4. Simonis, H., O’Sullivan, B.: Search strategies for rectangle packing. In: International Conference on Principles and Practice of Constraint Programming. pp. 52–66. Springer (2008)